Reparto de tareas — PetCare: Home → Mis Mascotas → Detalle → Registrar Cuidado (MVVM + Room + Hilt + Navigation Compose)

Contexto

Esta versión del plan incorpora las observaciones de una clase posterior del profesor (notas/transcripción) que **revierten dos restricciones anteriores** de este mismo plan, y agregan una corrección de arquitectura que no estaba cubierta:

1. **Se levanta la restricción de NavHost/NavController.** El profesor explica que ya no debe haber variables de navegación "quemadas" dentro de la UI (`remember { mutableStateOf(Pantalla.HOME) }` como lo teníamos en `PetCareApp.kt`), y que las rutas deben modelarse con `Enum` o clases selladas. La forma correcta de resolver esto es adoptar **Navigation Compose de verdad**, con rutas tipadas (`@Serializable sealed interface`), en vez de seguir con el `when` manual. Esto además resuelve solo, de forma gratuita, el problema de que el botón físico de "atrás" no funcionaba entre pantallas (que antes estaba documentado como "fuera de alcance").
2. **Se cambia de `AppContainer` manual a Hilt.** El profesor menciona explícitamente evitar el `lateinit var` de la inyección manual, y recomienda `@HiltViewModel` + `@Inject`. Se adopta Hilt para toda la inyección de dependencias (ViewModels, repositorios, base de datos).
3. **Nomenclatura "offline/local" para los repositorios.** `MascotaRepositoryImpl` → `OfflineMascotaRepository`, `CuidadoRepositoryImpl` → `OfflineCuidadoRepository`, para que el nombre deje claro que la fuente de datos es el almacenamiento local del dispositivo.

Estos dos cambios (Hilt + NavHost) en realidad **simplifican bastante el código del contenedor** (`PetCareApp.kt`), porque Hilt elimina toda la plomería manual de `viewModelFactory`/`AppContainer`, y Navigation Compose elimina las variables `mascotaIdSeleccionada`/`cuidadoIdSeleccionado` que antes vivían sueltas en el composable — ahora esos datos viajan como argumentos tipados de la ruta.

Lo demás del plan (3 pantallas + 1 contenedor, Room con 3 entidades — 2 maestras + 1 transaccional —, patrón Repositorio, MVVM con `StateFlow<UiState>` sellado, separación `Route`/`Screen`, eventos puntuales con `Channel`, Material 3) se mantiene igual que antes.

Supuestos (avísame si alguno no es lo que quieres)

- Se usa el sistema de **rutas tipadas con `kotlinx.serialization`** (`@Serializable` sobre una `sealed interface`), que es el estándar actual de Navigation Compose, en vez de rutas como `String` a mano.
- Hilt se usa para **ViewModels, Base de datos y Repositorios**. El resto del equipo sigue sin tocar Hilt directamente salvo por las anotaciones (`@HiltViewModel`, `@Inject constructor`) en sus propios archivos — la configuración "pesada" (módulos, Application class) la arma quien lidera el Paso 0.
- Se mantiene la interpretación de "2 tablas maestras + 1 transaccional" ya acordada: `Mascota` + `TipoCuidado` (maestras) + `Cuidado` (transaccional), y `Mascota` sigue siendo de solo lectura (precargada con Rocky).

Cómo esto encaja con la rúbrica de la Etapa 2 (actualizado)

Criterio de la rúbrica	¿Se cubre en esta entrega?
1. Capa de UI & State Hoisting (Compose, Material 3)	Sí — `Route` (stateful, obtiene su ViewModel con `hiltViewModel()`) / `Screen` (stateless), sin pasar ViewModel a hijos, `collectAsStateWithLifecycle()`.
2. Arquitectura MVVM & Gestión de Estado Reactivo	Sí — ViewModel por pantalla, `StateFlow<UiState>` sellado, eventos puntuales con `Channel`.
3. Capa de Datos (Room & Single Source of Truth)	Sí — 3 entidades (2 maestras + 1 transaccional), lecturas `Flow<T>`, escrituras `suspend fun`, CRUD completo sobre `Cuidado`.
4. Patrón Repositorio y Desacoplamiento	Sí — interfaces + `Offline*Repository`, nomenclatura que refleja la fuente local.
5. Navegación e Inyección de Dependencias (DI)	**Ahora sí, completo**: Navigation Compose con rutas tipadas + Hilt para toda la inyección.

Estructura de paquetes a crear

```
app/src/main/java/com/am/septimob/petcare/
■■■ PetCareApplication.kt              (NUEVO — @HiltAndroidApp)
■■■ PetCareRoute.kt                    (reemplaza Pantalla.kt — rutas tipadas con @Serializable)
■■■ PetCareApp.kt                      (Persona D — ahora solo arma el NavHost, sin estado propio)
■■■ data/
■   ■■■ Mascota.kt                     (@Entity — tabla maestra 1)
■   ■■■ TipoCuidado.kt                 (@Entity — tabla maestra 2)
■   ■■■ Cuidado.kt                     (@Entity — tabla transaccional)
■   ■■■ MascotaDao.kt
■   ■■■ TipoCuidadoDao.kt
■   ■■■ CuidadoDao.kt                  (CRUD completo)
■   ■■■ PetCareDatabase.kt             (@Database, sin el singleton manual — lo maneja Hilt)
■   ■■■ MascotaRepository.kt + OfflineMascotaRepository.kt   (renombrado, @Inject constructor)
■   ■■■ CuidadoRepository.kt + OfflineCuidadoRepository.kt   (renombrado, @Inject constructor)
■■■ di/
■   ■■■ DatabaseModule.kt              (NUEVO — @Module, provee Database + Daos)
■   ■■■ RepositoryModule.kt            (NUEVO — @Module @Binds, liga interfaces a Offline*Repository)
■■■ HomeScreen.kt + HomeViewModel.kt                          (Persona A — @HiltViewModel)
■■■ MascotasScreen.kt + MascotasViewModel.kt                  (Persona B — @HiltViewModel)
■■■ DetalleMascotaScreen.kt + DetalleMascotaViewModel.kt      (Persona C — @HiltViewModel + SavedStateH
■■■ RegistrarCuidadoScreen.kt + RegistrarCuidadoViewModel.kt  (Persona C — @HiltViewModel + SavedStateH
```

`AppContainer.kt` **se elimina** (ya no hace falta, Hilt reemplaza toda esa función).

Paso 0 — Base compartida (sesión conjunta del equipo)

0.1 — Dependencias en Gradle (creció: ahora incluye Hilt y Navigation Compose)

En el `build.gradle.kts` **raíz**, agregar los plugins (con `apply false`, igual que los que ya existen):

- `com.google.devtools.ksp` (KSP, para Room y para el compilador de Hilt).
- `org.jetbrains.kotlin.plugin.serialization` (para las rutas tipadas con `kotlinx.serialization`).
- `com.google.dagger.hilt.android` (Hilt).

En `gradle/libs.versions.toml`, agregar en `[versions]`, `[plugins]` y `[libraries]` las entradas para:

- Room: `room-runtime`, `room-ktx`, `room-compiler` (este último se agrega con `ksp(...)`).
- `androidx.navigation:navigation-compose`.
- `org.jetbrains.kotlinx:kotlinx-serialization-json`.
- `androidx.lifecycle:lifecycle-viewmodel-compose` (trae `collectAsStateWithLifecycle()` y `hiltViewModel()` en conjunto con la siguiente).
- Hilt: `com.google.dagger:hilt-android`, `com.google.dagger:hilt-android-compiler` (con `ksp(...)` — las versiones recientes de Hilt ya soportan KSP, no hace falta `kapt`).
- `androidx.hilt:hilt-navigation-compose` (trae la función `hiltViewModel()` para usar dentro de un `NavHost`).

En `app/build.gradle.kts`, aplicar los 3 plugins nuevos (sin `apply false`) y agregar las dependencias de arriba en el bloque `dependencies { }`.

Usar siempre la versión estable más reciente compatible con Kotlin `2.2.10` / Compose BOM `2026.02.01` — revisar en Android Studio, no copiar números a ciegas.

0.2 — `PetCareRoute.kt` (reemplaza a `Pantalla.kt`; no depende de nada)

```
package com.am.septimob.petcare

import kotlinx.serialization.Serializable

sealed interface PetCareRoute {
    @Serializable data object Home : PetCareRoute
    @Serializable data object Mascotas : PetCareRoute
    @Serializable data class Detalle(val mascotaId: Int) : PetCareRoute
    @Serializable data class RegistrarCuidado(val mascotaId: Int, val cuidadoId: Int? = null) : PetCareR
}
```

Cada ruta lleva sus propios argumentos tipados (`mascotaId`, `cuidadoId`) en vez de variables sueltas en el contenedor como antes.

0.3 — Contrato de datos (bloqueante: esto es lo primero que hay que subir)

```
// data/Mascota.kt
@Entity(tableName = "mascotas")
data class Mascota(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val nombre: String,
    val especie: String,
    val raza: String,
    val fechaNacimiento: String,
    val sexo: String,
    val pesoKg: Double
)

// data/TipoCuidado.kt — tabla maestra 2
@Entity(tableName = "tipos_cuidado")
data class TipoCuidado(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val nombre: String // "Vacunación", "Baño", "Desparasitación", "Visita veterinaria"
)

// data/Cuidado.kt — tabla transaccional (por eso necesita que Mascota y TipoCuidado ya existan)
@Entity(
```

```
    tableName = "cuidados",
    foreignKeys = [
        ForeignKey(entity = Mascota::class, parentColumns = ["id"], childColumns = ["mascotaId"], onDele
        ForeignKey(entity = TipoCuidado::class, parentColumns = ["id"], childColumns = ["tipoCuidadoId"]
    ],
    indices = [Index("mascotaId"), Index("tipoCuidadoId")]
)
data class Cuidado(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val mascotaId: Int,
    val tipoCuidadoId: Int,
    val fecha: String,
    val notas: String
)
```

```
// data/MascotaRepository.kt
interface MascotaRepository {
    fun obtenerMascotas(): Flow<List<Mascota>>
    fun obtenerMascotaPorId(id: Int): Flow<Mascota?>
}
```

```
// data/CuidadoRepository.kt — usa Cuidado y TipoCuidado en su firma, por eso va después
data class CuidadoConTipo(
    val id: Int,
    val fecha: String,
    val notas: String,
    val tipoCuidadoNombre: String
)

interface CuidadoRepository {
    fun obtenerCuidadosDeMascota(mascotaId: Int): Flow<List<CuidadoConTipo>>
    fun obtenerCuidadoPorId(id: Int): Flow<Cuidado?>
    fun obtenerTiposDeCuidado(): Flow<List<TipoCuidado>>
    suspend fun guardar(cuidado: Cuidado): Long
    suspend fun actualizar(cuidado: Cuidado)
    suspend fun eliminar(cuidado: Cuidado)
}
```

En cuanto existe este bloque, **las personas A, B y C ya pueden escribir sus ViewModels**, y los 2 sub-hilos de abajo pueden arrancar en paralelo.

0.4 — Sub-hilo 1: implementación de Mascota

```
// data/MascotaDao.kt
@Dao
interface MascotaDao {
    @Query("SELECT * FROM mascotas")
    fun obtenerTodas(): Flow<List<Mascota>>

    @Query("SELECT * FROM mascotas WHERE id = :id")
    fun obtenerPorId(id: Int): Flow<Mascota?>

    @Insert
    suspend fun insertar(mascota: Mascota)
}
```

```
// data/OfflineMascotaRepository.kt — @Inject constructor en vez de instanciarlo a mano
class OfflineMascotaRepository @Inject constructor(
    private val dao: MascotaDao
) : MascotaRepository {
    override fun obtenerMascotas(): Flow<List<Mascota>> = dao.obtenerTodas()
    override fun obtenerMascotaPorId(id: Int): Flow<Mascota?> = dao.obtenerPorId(id)
}
```

0.5 — Sub-hilo 2: implementación de TipoCuidado + Cuidado

```
// data/TipoCuidadoDao.kt
@Dao
interface TipoCuidadoDao {
    @Query("SELECT * FROM tipos_cuidado")
    fun obtenerTodos(): Flow<List<TipoCuidado>>

    @Insert
    suspend fun insertar(tipo: TipoCuidado)
}
```

```
// data/CuidadoDao.kt — CRUD completo (RF05-RF08)
@Dao
interface CuidadoDao {
    @Query("""
        SELECT cuidados.id AS id, cuidados.fecha AS fecha, cuidados.notas AS notas,
               tipos_cuidado.nombre AS tipoCuidadoNombre
        FROM cuidados
        INNER JOIN tipos_cuidado ON cuidados.tipoCuidadoId = tipos_cuidado.id
        WHERE cuidados.mascotaId = :mascotaId
        ORDER BY cuidados.fecha DESC
    """)
    fun obtenerCuidadosDeMascota(mascotaId: Int): Flow<List<CuidadoConTipo>>

    @Query("SELECT * FROM cuidados WHERE id = :id")
    fun obtenerPorId(id: Int): Flow<Cuidado?>

    @Insert
    suspend fun insertar(cuidado: Cuidado): Long

    @Update
    suspend fun actualizar(cuidado: Cuidado)

    @Delete
    suspend fun eliminar(cuidado: Cuidado)
}
```

```
// data/OfflineCuidadoRepository.kt
class OfflineCuidadoRepository @Inject constructor(
    private val cuidadoDao: CuidadoDao,
    private val tipoCuidadoDao: TipoCuidadoDao
) : CuidadoRepository {
    override fun obtenerCuidadosDeMascota(mascotaId: Int) = cuidadoDao.obtenerCuidadosDeMascota(mascotaI
    override fun obtenerCuidadoPorId(id: Int) = cuidadoDao.obtenerPorId(id)
    override fun obtenerTiposDeCuidado() = tipoCuidadoDao.obtenerTodos()
    override suspend fun guardar(cuidado: Cuidado) = cuidadoDao.insertar(cuidado)
    override suspend fun actualizar(cuidado: Cuidado) = cuidadoDao.actualizar(cuidado)
    override suspend fun eliminar(cuidado: Cuidado) = cuidadoDao.eliminar(cuidado)
}
```

0.6 — Unión: módulos de Hilt + Application (la hace quien lidera Persona D)

A diferencia de la versión anterior (con `AppContainer` + un singleton manual con `@Volatile`/`synchronized`), ahora **Hilt maneja el ciclo de vida de los singletons** — no hace falta escribir ese patrón a mano.

```
// data/PetCareDatabase.kt — ya no tiene companion object ni lógica de singleton
@Database(entities = [Mascota::class, TipoCuidado::class, Cuidado::class], version = 1, exportSchema = f
abstract class PetCareDatabase : RoomDatabase() {
    abstract fun mascotaDao(): MascotaDao
    abstract fun tipoCuidadoDao(): TipoCuidadoDao
    abstract fun cuidadoDao(): CuidadoDao
```

```
}
```

```
// di/DatabaseModule.kt
@Module
@InstallIn(SingletonComponent::class)
object DatabaseModule {

    @Provides
    @Singleton
    fun providePetCareDatabase(
        @ApplicationContext context: Context,
        mascotaDaoProvider: Provider<MascotaDao>,
        tipoCuidadoDaoProvider: Provider<TipoCuidadoDao>
    ): PetCareDatabase =
        Room.databaseBuilder(context, PetCareDatabase::class.java, "petcare.db")
            .addCallback(object : RoomDatabase.Callback() {
                override fun onCreate(db: SupportSQLiteDatabase) {
                    super.onCreate(db)
                    // Precarga: Rocky + el catálogo fijo de tipos de cuidado.
                    // Se usan Provider<T> porque el Dao depende de la Database que se
                    // está construyendo aquí mismo — Hilt resuelve esta referencia perezosa.
                    CoroutineScope(Dispatchers.IO).launch {
                        mascotaDaoProvider.get().insertar(
                            Mascota(nombre = "Rocky", especie = "Perro", raza = "Mestizo",
                                fechaNacimiento = "12/03/2021", sexo = "Macho", pesoKg = 18.5)
                        )
                        listOf("Vacunación", "Baño", "Desparasitación", "Visita veterinaria").forEach {
                            tipoCuidadoDaoProvider.get().insertar(TipoCuidado(nombre = it))
                        }
                    }
                }
            })
            .build()

    @Provides fun provideMascotaDao(db: PetCareDatabase): MascotaDao = db.mascotaDao()
    @Provides fun provideTipoCuidadoDao(db: PetCareDatabase): TipoCuidadoDao = db.tipoCuidadoDao()
    @Provides fun provideCuidadoDao(db: PetCareDatabase): CuidadoDao = db.cuidadoDao()
}
```

```
// di/RepositoryModule.kt
@Module
@InstallIn(SingletonComponent::class)
abstract class RepositoryModule {

    @Binds
    @Singleton
    abstract fun bindMascotaRepository(impl: OfflineMascotaRepository): MascotaRepository

    @Binds
    @Singleton
    abstract fun bindCuidadoRepository(impl: OfflineCuidadoRepository): CuidadoRepository
}
```

```
// PetCareApplication.kt
@HiltAndroidApp
class PetCareApplication : Application()
```

Y en `AndroidManifest.xml`, en la etiqueta `<application>`, agregar `android:name=".PetCareApplication"`.

```
// MainActivity.kt
@AndroidEntryPoint
class MainActivity : ComponentActivity() {
```

```
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            PetCareTheme {
                PetCareApp()
            }
        }
    }
}
```

Nótese que `MainActivity` ya **no crea nada a mano** (ni `AppContainer` ni base de datos) — todo lo resuelve Hilt automáticamente gracias a `@AndroidEntryPoint` + `@HiltAndroidApp`.

Paso 1 — Trabajo en paralelo (una persona por pantalla; Persona C tiene 2 pantallas)

Regla estricta: el composable `XRoute` es el único que conoce el ViewModel (lo obtiene con `hiltViewModel()` como valor por defecto de un parámetro, no recibido desde el contenedor). El composable `XScreen` solo recibe el `UiState` + lambdas. **Usar componentes Material 3** en todas las pantallas (`Scaffold`, `TopAppBar`, `Card`, `OutlinedTextField`, `CircularProgressIndicator`, colores de `MaterialTheme.colorScheme`).

Persona A — `HomeScreen.kt` + `HomeViewModel.kt`

```
// HomeViewModel.kt
sealed interface HomeUiState {
    data object Loading : HomeUiState
    data class Success(val cantidadMascotas: Int) : HomeUiState
    data class Error(val mensaje: String) : HomeUiState
}

@HiltViewModel
class HomeViewModel @Inject constructor(
    repository: MascotaRepository
) : ViewModel() {
    val uiState: StateFlow<HomeUiState> = repository.obtenerMascotas()
        .map { mascotas -> HomeUiState.Success(mascotas.size) as HomeUiState }
        .catch { emit(HomeUiState.Error(it.message ?: "Error al cargar")) }
        .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000), HomeUiState.Loading)
}
```

```
// HomeScreen.kt
@Composable
fun HomeRoute(
    onVerMascotasClick: () -> Unit,
    viewModel: HomeViewModel = hiltViewModel()
) {
    val uiState by viewModel.uiState.collectAsStateWithLifecycle()
    HomeScreen(uiState = uiState, onVerMascotasClick = onVerMascotasClick)
}

@Composable
fun HomeScreen(uiState: HomeUiState, onVerMascotasClick: () -> Unit) {
    when (uiState) {
        is HomeUiState.Loading -> // indicador de carga
        is HomeUiState.Success -> // resumen con uiState.cantidadMascotas + botón "Ver mis mascotas"
        is HomeUiState.Error -> // mensaje de error
    }
}

@Preview
```

```
@Composable
private fun HomeScreenPreview() {
    HomeScreen(uiState = HomeUiState.Success(cantidadMascotas = 1), onVerMascotasClick = {})
}
```

Wireframe "Pantalla 2 — Home".

Persona B — `MascotasScreen.kt` + `MascotasViewModel.kt`

Mismo patrón que Home (`@HiltViewModel`, `MascotasRoute` con `hiltViewModel()` por defecto). `MascotasUiState` (`Loading`/`Success(mascotas)`/`Error`). Wireframe "Pantalla 3 — Mis mascotas".

Persona C — `DetalleMascotaScreen.kt` + `DetalleMascotaViewModel.kt` + `RegistrarCuidadoScreen.kt` + `RegistrarCuidadoViewModel.kt`

Esta persona tiene el trabajo más grande — si Persona A o B terminan antes, pueden ayudar aquí.

Diferencia clave respecto a la versión anterior: antes `mascotaId`/`cuidadoId` llegaban como parámetro de constructor armado a mano con `viewModelFactory`. Ahora llegan automáticamente vía `SavedStateHandle`, porque Navigation Compose mete los argumentos tipados de la ruta ahí solo:

```
sealed interface DetalleUiState {
    data object Loading : DetalleUiState
    data class Success(val mascota: Mascota, val cuidados: List<CuidadoConTipo>) : DetalleUiState
    data class Error(val mensaje: String) : DetalleUiState
}

@HiltViewModel
class DetalleMascotaViewModel @Inject constructor(
    savedStateHandle: SavedStateHandle,
    mascotaRepository: MascotaRepository,
    cuidadoRepository: CuidadoRepository
) : ViewModel() {
    private val ruta = savedStateHandle.toRoute<PetCareRoute.Detalle>()

    val uiState: StateFlow<DetalleUiState> =
        combine(
            mascotaRepository.obtenerMascotaPorId(ruta.mascotaId),
            cuidadoRepository.obtenerCuidadosDeMascota(ruta.mascotaId)
        ) { mascota, cuidados ->
            if (mascota != null) DetalleUiState.Success(mascota, cuidados)
            else DetalleUiState.Error("Mascota no encontrada")
        }
            .catch { emit(DetalleUiState.Error(it.message ?: "Error al cargar")) }
            .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000), DetalleUiState.Loading)
}
```

`DetalleMascotaRoute(onRegistrarCuidadoClick, onCuidadoClick, onBackClick, viewModel: DetalleMascotaViewModel = hiltViewModel())` dibuja los datos de la mascota (wireframe "Pantalla 5") + la lista de cuidados (wireframe "Pantalla 6") + botón "Registrar cuidado".

```
sealed interface RegistrarCuidadoUiState {
    data object Cargando : RegistrarCuidadoUiState
    data class Formulario(
        val tiposCuidado: List<TipoCuidado>,
        val tipoCuidadoSeleccionadoId: Int? = null,
        val fecha: String = "",
        val notas: String = "",
        val esEdicion: Boolean = false
    ) : RegistrarCuidadoUiState
    data class Error(val mensaje: String) : RegistrarCuidadoUiState
```

```
}

sealed interface RegistrarCuidadoEvento {
    data object GuardadoExitoso : RegistrarCuidadoEvento
    data object EliminadoExitoso : RegistrarCuidadoEvento
    data class Error(val mensaje: String) : RegistrarCuidadoEvento
}

@HiltViewModel
class RegistrarCuidadoViewModel @Inject constructor(
    savedStateHandle: SavedStateHandle,
    private val repository: CuidadoRepository
) : ViewModel() {
    private val ruta = savedStateHandle.toRoute<PetCareRoute.RegistrarCuidado>()

    private val _eventos = Channel<RegistrarCuidadoEvento>()
    val eventos = _eventos.receiveAsFlow()

    // uiState: carga tiposCuidado (y el cuidado existente si ruta.cuidadoId != null) y arma Formulario

    fun guardar(tipoCuidadoId: Int, fecha: String, notas: String) {
        viewModelScope.launch {
            try {
                repository.guardar(Cuidado(id = ruta.cuidadoId ?: 0, mascotaId = ruta.mascotaId,
                    tipoCuidadoId = tipoCuidadoId, fecha = fecha, notas = notas))
                _eventos.send(RegistrarCuidadoEvento.GuardadoExitoso)
            } catch (e: Exception) {
                _eventos.send(RegistrarCuidadoEvento.Error(e.message ?: "No se pudo guardar"))
            }
        }
    }

    fun eliminar() {
        viewModelScope.launch {
            // repository.eliminar(...) del cuidado actual, luego:
            _eventos.send(RegistrarCuidadoEvento.EliminadoExitoso)
        }
    }
}
```

`RegistrarCuidadoRoute` colecta `uiState` **y** escucha `eventos` con `LaunchedEffect` para mostrar un Snackbar y volver cuando llega `GuardadoExitoso`/`EliminadoExitoso` — es el patrón de "eventos puntuales con canales" de la rúbrica. Formulario con `OutlinedTextField` + `ExposedDropdownMenuBox` (Material 3). Wireframes "Pantalla 7" y "Pantalla 8".

Persona D — `PetCareApp.kt` (ahora mucho más simple: solo el grafo de navegación)

```
@Composable
fun PetCareApp(navController: NavHostController = rememberNavController()) {
    NavHost(navController = navController, startDestination = PetCareRoute.Home) {
        composable<PetCareRoute.Home> {
            HomeRoute(onVerMascotasClick = { navController.navigate(PetCareRoute.Mascotas) })
        }

        composable<PetCareRoute.Mascotas> {
            MascotasRoute(
                onMascotaClick = { id -> navController.navigate(PetCareRoute.Detalle(mascotaId = id)) },
                onBackClick = { navController.popBackStack() }
            )
        }
        composable<PetCareRoute.Detalle> { backStackEntry ->
```

```
                val ruta = backStackEntry.toRoute<PetCareRoute.Detalle>()
                DetalleMascotaRoute(
                    onRegistrarCuidadoClick = {
                        navController.navigate(PetCareRoute.RegistrarCuidado(mascotaId = ruta.mascotaId))
                    },
                    onCuidadoClick = { cuidadoId ->
                        navController.navigate(PetCareRoute.RegistrarCuidado(mascotaId = ruta.mascotaId, cui
                    },
                    onBackClick = { navController.popBackStack() }
                )
            }

            composable<PetCareRoute.RegistrarCuidado> {
                RegistrarCuidadoRoute(onListo = { navController.popBackStack() })
            }
        }
    }
```

Notas para esta persona:

- **Ya no hay estado local en este composable** (ni `pantallaActual`, ni `mascotaIdSeleccionada`, ni `cuidadoIdSeleccionado`) — todo lo maneja `NavController` y las rutas tipadas. Esto es justo lo que pedía la corrección del profesor.
- **Ya no hay** `viewModelFactory` **ni** `AppContainer` — cada `XRoute` obtiene su propio ViewModel con `hiltViewModel()`.
- El botón físico/gesto de "atrás" del sistema **ahora sí funciona solo**, gratis, porque lo da `NavHost`/`NavController` — ya no es una limitación de esta entrega.
- Esta persona también deja lista la parte de Hilt (Paso 0.6: módulos + `PetCareApplication` + `AndroidManifest.xml`).

Paso 2 — Integración y verificación (todo el equipo)

1. Cada persona sube su(s) archivo(s) a su propia rama y abre PR contra `main`.
2. Se integra en orden: `data/` completo + `di/` + `PetCareApplication.kt` + `PetCareRoute.kt`, luego las pantallas, luego `PetCareApp.kt` + `MainActivity.kt` + `AndroidManifest.xml`.
3. Compilar y correr. Probar a mano:

- Flujo completo: Home → Mascotas → Detalle de Rocky → Registrar cuidado (elegir tipo, fecha, notas) → guardar → aparece al instante en la lista → editarlo → eliminarlo.
- **Botón físico de "atrás" del sistema** en cada pantalla — debe regresar correctamente por la pila de navegación (esto es nuevo, antes no se podía probar).
- **Offline-first**: cerrar la app por completo (matar el proceso), activar modo avión, volver a abrirla — todo lo que no se borró debe seguir ahí.

1. Revisar que ningún `XScreen` reciba el ViewModel directamente.
2. Revisar que ningún ViewModel ni Repository se instancie con `new`/constructor directo en ningún lado — todo debe venir de Hilt (`@Inject`, `@Provides`, `@Binds`).

Recordatorio: presentación de cierre (elevator pitch, 5 minutos)

- **1 min** — nombre de la app y el problema que resuelve.
- **2 min** — demo en vivo: registrar/editar/eliminar un cuidado, navegación con el botón físico de atrás, y la prueba de offline-first (cerrar app + modo avión + reabrir).
- **2 min** — arquitectura: `Route`/`Screen` sin pasar el ViewModel, `StateFlow`/`Flow` desde Room hasta la UI, **y cómo Hilt inyecta el repositorio y el ViewModel sin instanciarlos a mano** (mostrar un `@Module` y un `@HiltViewModel`).
- Los 4 integrantes deben poder responder preguntas de cualquier capa.

Fuera de alcance de esta entrega (a propósito)

- **Registrar/editar/eliminar Mascota** — sigue de solo lectura, precargada con Rocky. El CRUD completo es sobre `Cuidado`.
- DataStore, Splash Screen, pantalla de Configuración — siguen pendientes.
- Casos de uso (`UseCases`) de la capa de dominio — extra opcional del documento (+0.3 pts), la lógica de cada ViewModel sigue siendo simple.
- Ícono adaptativo vectorizado (+0.3 pts) y `WindowSizeClass` (+0.4 pts) — extras opcionales.
- Modelos de IA on-device — mencionados como posibilidad por el profesor para quien le interese, no aplica al alcance de este equipo por ahora.